SUPER MARIO

AI Foundations Case Study

Using BFS | A* Search | Genetic Algorithm | PSO | Bayesian Network | MDP

Case Domain: Super Mario Bros — An AI Adventure

Mario is supported by an experimental AI Adventure Assistant that observes the level, evaluates risk, searches for routes, predicts enemy behaviour, manages power-ups, and helps choose optimal actions. This case study explores seven fundamental AI algorithms applied to real in-game scenarios.

Submitted By	Siddhant Kumar Singh
Registration No	2024109296
Course	Artificial Intelligence
Course Code	CS / AI — Undergraduate
Submission Date	November 2026
Under Guidance of	Faculty, Department of Computer Science

Greedy BFS Chase the Super Star	**BFS / A*** Compare shortest routes	**Genetic Algorithm** Evolve Mario's Policy	**PSO** Tune behaviour weights

Table of Contents

1. **Introduction**
    - 1.1 Overview of Case Study
    - 1.2 Objectives
    - 1.3 Algorithm-to-Task Mapping
2. **Tasks and Solutions**
    - 2.1 Greedy Best-First Search — Chase the Super Star
    - 2.2 Genetic Algorithm — Evolve a High-Scoring Policy
    - 2.3 Bayesian Network — Infer Hidden Threats
    - 2.4 Markov Decision Process — Choose Optimal Action
    - 2.5 Breadth-First Search — Guarantee the Shortest Route
    - 2.6 A* Search — Balance Cost and Distance
    - 2.7 Particle Swarm Optimization — Tune Mario's Policy
3. **Comparative Analysis & Discussion**
4. **Conclusion**
5. **References**

1. Introduction

1.1 Overview of Case Study

This case study explores the application of seven fundamental Artificial Intelligence algorithms to the iconic Super Mario Bros game environment. Rather than treating AI algorithms as abstract mathematical concepts, we embed them inside a rich, relatable narrative where Mario navigates a dynamic Mushroom Kingdom filled with enemies, power-ups, hidden threats, and time-sensitive objectives.

The Super Mario world provides an ideal testing ground for AI methods because it naturally contains search problems (navigating between locations), optimisation problems (finding the best playing strategy), uncertainty (hidden enemies and power-ups), and sequential decision-making (choosing actions with long-term consequences).

1.2 Objectives

- Apply Greedy Best-First Search (GBFS) to quickly navigate toward a time-sensitive Super Star before it disappears.
- Use a Genetic Algorithm (GA) to evolve a general Mario behaviour policy that performs well across multiple level layouts.
- Apply a Bayesian Network to estimate the probability of hidden threats (Piranha Plants, Bullet Bill launchers, hidden power-ups) from observable clues.
- Use a Markov Decision Process (MDP) solved via Value Iteration to determine the optimal action for Mario given his current belief state.
- Understand the strengths, limitations, and complementary roles of each algorithm within the same game environment.

1.3 Algorithm-to-Task Mapping

Algorithm	Super Mario Task	AI Concept Demonstrated
Greedy Best-First Search	Chase a Disappearing Super Star	Heuristic-guided fast search
Breadth-First Search	Find the shortest safe route	Uninformed complete search
A* Search	Find an efficient optimal route	Cost-aware heuristic search
Genetic Algorithm	Evolve a High-Scoring Level-Completion Policy	Evolutionary optimisation, fitness landscape
Particle Swarm Optimization	Tune Mario's behaviour weights	Population-based continuous optimisation
Bayesian Network	Infer Hidden Threats from Observable Clues	Probabilistic reasoning under uncertainty
Markov Decision Process	Choose the Best Action Given Belief State	Sequential decision-making, Value Iteration

2. Tasks and Solutions

2.1. Greedy Best-First Search — Chase the Disappearing Super Star

Explanation of the Task (Narrative)

A Super Star has appeared on a distant platform inside Bowser's castle. The star will vanish after only a short period — Mario has very limited time to calculate a perfect route. He needs to move quickly toward whichever platform brings him closest to the star. His AI assistant applies Greedy Best-First Search, always expanding the node that LOOKS closest to the star based on a Manhattan-distance heuristic. This makes decisions fast, but as Mario discovers, the apparently nearest location sometimes leads into a dead-end, forcing a detour. The lesson: a good estimate can guide fast decisions, but estimates can also be misleading.

PEAS Description

PEAS Component	Description
Performance	Reach the Super Star before the countdown timer expires; minimize moves taken.
Environment	2-D castle grid (6x7) with walls (#), open floors (.), start (S), and goal (G).
Actuators	Move Up, Move Down, Move Left, Move Right — one cell per action.
Sensors	Current (row, col) position, goal position, grid layout, h(n) values.

Task Environment Properties

Property	Value	Justification
Observability	Fully Observable	Mario and AI can see the entire grid at all times.
Determinism	Deterministic	Every move has a guaranteed, predictable outcome.
Episodicity	Sequential	Each move affects future moves and path options.
Dynamism	Static	Walls and goal do not change during the search.
Continuity	Discrete	Grid cells are discrete; actions are finite.
Agent Count	Single-Agent	Only Mario navigates; no adversary involved.

Problem Formulation

State	(row, col) — integer grid coordinates of Mario's position
Initial State	(0, 0) — top-left corner of the castle section
Goal State	(5, 6) — cell containing the Super Star
Actions	UP, DOWN, LEFT, RIGHT — move to adjacent non-wall cell
Heuristic h(n)	Manhattan Distance: |row_n - row_goal| + |col_n - col_goal|
Data Structure	Min-heap priority queue ordered by h(n)

Explanation of Algorithm Utilized

Greedy Best-First Search (GBFS) is an informed search algorithm that always expands the node that appears closest to the goal according to a heuristic function h(n). Unlike A* which combines path cost g(n) and h(n), GBFS uses **only h(n)** for node selection, making it faster but sacrificing optimality.

The algorithm maintains a priority queue (min-heap) where each entry is (h(n), node, path). At each step, it pops the node with the smallest h(n), marks it visited, and expands its unvisited neighbours. This continues until the goal is reached or the queue is empty.

Properties: Complete — Not always (can loop without visited tracking). Optimal — No (may find longer path than BFS). Time Complexity — O(b^m) worst case. Space Complexity — O(b^m). However, with a good heuristic, practical performance is much better.

How the Algorithm is Applied to this Task

The castle section is represented as a 6×7 binary grid where 0 = open floor and 1 = wall. Mario starts at (0,0) and the Super Star is at (5,6).

At each expansion step, the AI computes h(n) = Manhattan distance to goal for every unvisited neighbour and inserts them into the priority queue. The node with the smallest h(n) is always selected next — this greedily guides Mario toward the star without exhaustively exploring all paths.

The algorithm tracks a visited set to avoid revisiting cells. Once (5,6) is reached, the path is reconstructed from the parent map. If a dead-end is encountered (lower h but blocked), the algorithm backtracks via the heap and tries the next best option — demonstrating GBFS's weakness with misleading heuristics.

Code

greedy_bfs.py / genetic_algorithm.py / bayesian_network.py / markov_decision_process.py

```
import heapq
GRID = [ # 0=open, 1=wall
[0, 0, 1, 0, 0, 0, 0],
[0, 1, 1, 0, 1, 0, 0],
[0, 0, 0, 0, 1, 0, 1],
[1, 0, 1, 0, 0, 0, 0],
[0, 0, 0, 1, 0, 1, 0],
[0, 1, 0, 0, 0, 0, 0],
]
START, GOAL = (0,0), (5,6)
def heuristic(pos, goal): # Manhattan Distance
return abs(pos[0]-goal[0]) + abs(pos[1]-goal[1])
def greedy_bfs(grid, start, goal):
open_list = [(heuristic(start,goal), start, [start])]
heapq.heapify(open_list)
visited = {start}
while open_list:
h_val, current, path = heapq.heappop(open_list)
if current == goal: return path, True
for dr,dc in [(-1,0),(1,0),(0,-1),(0,1)]:
nr,nc = current[0]+dr, current[1]+dc
if 0<=nr<ROWS and 0<=nc<COLS and grid[nr][nc]==0:
if (nr,nc) not in visited:
visited.add((nr,nc))
heapq.heappush(open_list,
(heuristic((nr,nc),goal),(nr,nc),path+[(nr,nc)]))
```

```
    return [], False
```

Sample Output

Console Output

```
============================================================
Greedy Best-First Search — Chasing the Super Star
============================================================
Start : (0, 0) Goal : (5, 6)
============================================================
Step Current h(n) Neighbors Considered
------------------------------------------------------------
1 (0, 0) 11 DOWN:(1,0)(h=10), RIGHT:(0,1)(h=10)
2 (0, 1) 10 LEFT:(0,0)(visited)
3 (1, 0) 10 UP:(0,0)(visited), DOWN:(2,0)(h=9)
... ... .. ...
13 (5, 6) 0 UP:(4,6)(visited), LEFT:(5,5)(h=1)
============================================================
Super Star REACHED in 13 expansions!
Path length : 11 moves
============================================================
Grid: S . # . . . .
. # # . # . .
* * * * # . #
# . # * * * *
. . . # . # *
. # . . . . G
```

Key Observations

- GBFS always expands the node that LOOKS closest to the goal — it is fast but not optimal.
- In 13 expansions, it found an 11-move path. BFS would guarantee the shortest path but explore more nodes.
- If the heuristic is misleading (apparent closer cell is actually behind a wall), GBFS may take a longer detour.
- GBFS is ideal for time-sensitive scenarios like chasing a disappearing Star where speed > optimality.
- Adding a visited set makes it complete (no infinite loops) but still not optimal.

2.2. Genetic Algorithm — Evolve a High-Scoring Level-Completion Policy

Explanation of the Task (Narrative)

Mario needs more than a one-time path — he needs a reusable behaviour policy that works across many different level layouts. Instead of manually tuning his behavioral tendencies, an experimental AI system evolves a population of 'Mario personalities'. Each chromosome encodes seven behavioral weights: coin attraction, power-up attraction, enemy avoidance, jump aggressiveness, goal urgency, retreat tendency, and open-space preference. A chromosome with very high coin attraction collects many coins but enters dangerous regions frequently. One with high enemy avoidance survives longer but makes little progress. The genetic algorithm finds a balanced chromosome through selection, crossover, and mutation over 30 generations.

PEAS Description

PEAS Component	Description
Performance	Level completion rate, coins collected, remaining lives, time taken, power-ups, survival rate.
Environment	Simulated Super Mario level with coins, enemies, power-ups, pipes, and a goal flag.
Actuators	Behavioral weights [0,1] that guide action selection at each game tick.
Sensors	Level state, proximity to coins/enemies/goal, power-up availability, lives remaining.

Task Environment Properties

Property	Value	Justification
Observability	Partially Observable	Mario cannot see all future enemy positions.
Determinism	Stochastic	Enemy patterns and outcomes have random elements.
Episodicity	Sequential	Each generation depends on results from the prior one.
Dynamism	Dynamic	Level state changes with enemy movement and collectibles.
Continuity	Continuous	Behavioral weights are continuous-valued [0.0, 1.0].
Agent Count	Single-Agent	One Mario controller is evaluated at a time.

Problem Formulation

Chromosome	[coin_wt, powerup_wt, enemy_avoid_wt, jump_wt, goal_wt, retreat_wt, space_wt]
Gene Range	Each gene: float in [0.0, 1.0]
Fitness Function	500*completion + 0.5*coins + 2*powerups + 30*survival + 50*time_bonus

Selection	Tournament Selection (k=3 contenders)
Crossover	Single-point crossover, rate = 0.80
Mutation	Gaussian perturbation, rate = 0.15, strength = 0.20
Elitism	Top 2 chromosomes preserved unchanged each generation
Population	20 chromosomes, evolved over 30 generations

Explanation of Algorithm Utilized

A Genetic Algorithm (GA) is an evolutionary optimisation method inspired by biological natural selection. It maintains a **population** of candidate solutions (chromosomes), evaluates each with a **fitness function**, and iteratively produces better generations through three operators: **Selection** (fitter individuals survive), **Crossover** (two parents exchange genetic material to produce offspring), and **Mutation** (small random changes maintain diversity).

Unlike hill-climbing, GAs explore multiple regions of the search space simultaneously, making them robust against local optima. The fitness landscape for Mario policy optimisation is complex — high coin attraction alone creates a locally good but globally poor policy (never finishes levels), so the fitness function must carefully balance multiple objectives.

Key Operators: Tournament Selection randomly picks k=3 individuals and selects the fittest. Single-point crossover exchanges genes after a random split point. Gaussian mutation perturbs each gene with a small random delta, ensuring values stay within [0,1]. Elitism preserves the top 2 chromosomes to prevent regression.

How the Algorithm is Applied to this Task

Each chromosome represents a complete Mario playing style encoded as 7 continuous weights. The fitness simulator evaluates how well that style performs: completion probability (driven by goal urgency and enemy avoidance), coins collected (driven by coin attraction, penalised by dying), power-ups gathered, survival score, and time efficiency.

Over 30 generations, the population evolves from random personalities toward balanced controllers. Early generations show high variance — some chromosomes are purely aggressive (high coin attraction, low enemy avoidance) and die frequently. Later generations converge on balanced strategies with high GoalUrgency, high EnemyAvoidance, and moderate CoinAttraction.

The best evolved chromosome after 30 generations achieved a fitness of 680.00, with JumpAggressiveness deliberately kept low (0.169) despite other traits being maximised — showing the algorithm correctly learned that excessive jumping without open-space awareness is dangerous.

Code

greedy_bfs.py / genetic_algorithm.py / bayesian_network.py / markov_decision_process.py

```
import random, math
random.seed(42)
POPULATION_SIZE=20; CHROMOSOME_LEN=7; GENERATIONS=30
CROSSOVER_RATE=0.8; MUTATION_RATE=0.15
GENE_NAMES = ['CoinAttraction','PowerUpAttraction',
'EnemyAvoidance','JumpAggressiveness',
'GoalUrgency','RetreatTendency','OpenSpacePreference']
def simulate_mario(chromosome): # fitness function
coin_wt,powerup_wt,enemy_wt,jump_wt,goal_wt,retreat_wt,space_wt = chromosome
completion = 0.5*goal_wt + 0.3*enemy_wt - 0.2*max(0,jump_wt-space_wt)
```

```
completion = max(0.0, min(1.0, completion))
coins = coin_wt * 100 * (1 - 0.4*(1-enemy_wt))
fitness = completion*500 + coins*0.5 + powerup_wt*40 + ...
return round(fitness, 2)
def tournament_select(population, fitnesses, k=3):
contenders = random.sample(list(zip(population,fitnesses)), k)
return max(contenders, key=lambda x: x[1])[0][:]
def crossover(p1, p2): # single-point
if random.random() < CROSSOVER_RATE:
pt = random.randint(1, CHROMOSOME_LEN-1)
return p1[:pt]+p2[pt:], p2[:pt]+p1[pt:]
return p1[:], p2[:]
def mutate(c): # Gaussian perturbation
for i in range(CHROMOSOME_LEN):
if random.random() < MUTATION_RATE:
c[i] = max(0.0, min(1.0, c[i]+random.gauss(0,0.2)))
return c
```

Sample Output

Console Output

```
================================================================
Genetic Algorithm — Evolving Mario's Behaviour Policy
================================================================
Population: 20 | Genes: 7 | Generations: 30
================================================================
Gen Best Fitness Avg Fitness Best Chromosome (first 4 genes)
----------------------------------------------------------------
1 462.03 310.91 ['0.21', '0.94', '0.88', '0.32']
5 613.04 562.76 ['0.11', '0.91', '1.00', '0.17']
10 647.15 627.20 ['0.38', '1.00', '1.00', '0.17']
20 675.85 659.22 ['0.92', '1.00', '1.00', '0.17']
30 680.00 669.16 ['1.00', '1.00', '1.00', '0.17']
================================================================
EVOLUTION COMPLETE | Best Fitness: 680.00
================================================================
Best Mario Policy:
CoinAttraction : 1.000 [####################]
PowerUpAttraction : 1.000 [####################]
EnemyAvoidance : 1.000 [####################]
JumpAggressiveness : 0.169 [### ]
GoalUrgency : 1.000 [####################]
```

Key Observations

- The GA evolved a balanced policy — maximising GoalUrgency + EnemyAvoidance while keeping JumpAggressiveness low.
- Fitness improved from 462 (Gen 1) to 680 (Gen 30) — a 47% improvement through evolution.
- Pure coin-chasing policies (high CoinAttraction alone) plateau early because they die frequently.
- Elitism prevents the best solution from being lost through crossover or mutation.
- The fitness function design is critical: wrong weights can reward undesirable behaviour (e.g., coins > completion).
- The algorithm discovers that JumpAggressiveness without OpenSpacePreference is dangerous — a non-obvious insight.

2.3. Bayesian Network — Infer Hidden Threats from Observable Clues

Explanation of the Task (Narrative)

Mario approaches a partially hidden section of Bowser's castle. A Piranha Plant may emerge from a pipe, a Bullet Bill launcher may be active, and a hidden block may contain a power-up — but Mario cannot see these directly. He has only two observable clues: whether he hears a warning sound (SoundCue) and whether the section is partially visible (ScreenVisibility). His AI assistant constructs a Bayesian Network to estimate the probability of each hidden threat. When both sound and partial visibility are detected, the probability of a Piranha Plant threat jumps to 72% — and the recommended action shifts from ADVANCE to WAIT or RETREAT. This separates the task of BELIEF ESTIMATION from the task of ACTION SELECTION (handled by the MDP in the next section).

PEAS Description

PEAS Component	Description
Performance	Correctly estimate threat probabilities; avoid being hit; collect power-up when safe.
Environment	Partially observable castle section with hidden Piranha Plants, Bullet Bill launchers, and power-up blocks.
Actuators	Output: probability estimates for PipeThreat, LauncherActive, HiddenPowerUp.
Sensors	SoundCue (True/False) — audible warning; ScreenVisibility (True/False) — partial visual access.

Task Environment Properties

Property	Value	Justification
Observability	Partially Observable	PipeThreat, LauncherActive, HiddenPowerUp are hidden; only clues observable.
Determinism	Stochastic	Enemy emergence and power-up presence are probabilistic.
Episodicity	Sequential	Evidence updates belief; updated belief feeds into action selection.
Dynamism	Dynamic	Threats may appear or disappear as Mario moves.
Continuity	Discrete	Variables are binary (True/False); probabilities are continuous.
Agent Count	Single-Agent	Only Mario is reasoning about threats.

Problem Formulation

Network Variables	PipeThreat, LauncherActive, HiddenPowerUp (hidden); SoundCue, ScreenVis (observed)
Prior P(PipeThreat)	P(True)=0.35, P(False)=0.65
Likelihood (Sound)	P(Sound=T|PipeThreat=T)=0.80; P(Sound=T|PipeThreat=F)=0.25

Likelihood (Screen)	P(Screen=T|PipeThreat=T)=0.60; P(Screen=T|PipeThreat=F)=0.40
Bayes' Rule	P(PipeThreat|evidence) ∝ P(PipeThreat) × P(Sound|PT) × P(Screen|PT)
Child Variables	P(LauncherActive|PT) and P(HiddenPowerUp|PT) computed via marginalization

Explanation of Algorithm Utilized

A Bayesian Network (BN) is a directed acyclic graph (DAG) where nodes represent random variables and edges encode conditional dependencies. Each node stores a conditional probability table (CPT) giving P(node | parents).

The core operation is **Bayesian inference**: given observed evidence E, compute the posterior probability of hidden variables H using Bayes' Rule: P(H|E) = P(E|H) × P(H) / P(E). The denominator P(E) is a normalisation constant ensuring probabilities sum to 1.

In this case study, PipeThreat is the root node influenced by observable clues (SoundCue, ScreenVisibility). LauncherActive and HiddenPowerUp are child nodes whose probabilities depend on PipeThreat. When evidence arrives (e.g., sound heard AND screen partially visible), the posterior for PipeThreat updates from its prior of 0.35 to 0.72 — a dramatic change that alters Mario's recommended action.

How the Algorithm is Applied to this Task

Four observable scenarios are tested: (1) Quiet & No visibility, (2) Sound heard, no visibility, (3) Quiet but partial visibility, (4) Sound + partial visibility. For each scenario, the posterior probabilities of all hidden variables are computed using Bayes' Rule applied to SoundCue and ScreenVisibility.

The posteriors directly inform the risk assessment: if PipeThreat > 0.60 OR LauncherActive > 0.50, the recommendation is WAIT or RETREAT. If HiddenPowerUp > 0.40 with moderate threat, the recommendation is JUMP carefully to grab the power-up. Otherwise, ADVANCE.

These posterior probabilities then serve as the 'belief state' input to the Markov Decision Process, creating a complete Bayesian-MDP pipeline for reasoning and action under uncertainty.

Code

greedy_bfs.py / genetic_algorithm.py / bayesian_network.py / markov_decision_process.py

```
# Prior and conditional probabilities
P_pipe_threat = {True: 0.35, False: 0.65}
P_sound_given_pipe = {True: {True:0.80, False:0.20},
False:{True:0.25, False:0.75}}
P_screen_given_pipe = {True: {True:0.60, False:0.40},
False:{True:0.40, False:0.60}}
def posterior_pipe_threat(sound_cue, screen_vis):
scores = {}
for threat in [True, False]:
prior = P_pipe_threat[threat]
l_snd = P_sound_given_pipe[threat][sound_cue]
l_scr = P_screen_given_pipe[threat][screen_vis]
scores[threat] = prior * l_snd * l_scr # Bayes numerator
total = sum(scores.values())
return {k: round(v/total, 4) for k,v in scores.items()}
def estimate_all(sound_cue, screen_vis):
```

```
post_pipe = posterior_pipe_threat(sound_cue, screen_vis)
# Marginalize: P(LauncherActive) = sum_pipe T(la|pipe)*P(pipe|evidence)
p_launcher = {la: sum(P_launcher_given_pipe[p][la]*post_pipe[p]
for p in [True,False]) for la in [True,False]}
return {'PipeThreat':post_pipe, 'LauncherActive':p_launcher, ...}
```

Sample Output

Console Output

```
================================================================
BAYESIAN NETWORK — Super Mario Threat Estimation
================================================================
=== Scenario: Sound + partial visibility ===
Evidence: SoundCue=True, ScreenVisibility=True
----------------------------------------------------------------
Piranha Plant Emerging from Pipe
P(PipeThreat=True ) = 0.7210 [###################### ]
P(PipeThreat=False) = 0.2790 [######## ]

Bullet Bill Launcher is Active
P(LauncherActive=True ) = 0.5605 [################# ]
P(LauncherActive=False) = 0.4395 [############# ]

Block Contains Hidden Power-Up
P(HiddenPowerUp=True ) = 0.4803 [############## ]

Risk Assessment:
=> HIGH THREAT (Pipe:0.72, Launcher:0.56)
=> RECOMMENDATION: WAIT or RETREAT
```

Key Observations

- Bayesian Networks allow Mario to reason about UNSEEN threats using observed clues.
- Prior P(PipeThreat)=0.35 updates to 0.72 when both sound and visibility clues are active.
- A 20% chance of a lethal threat may still justify waiting — consequences matter as much as probability.
- The Bayesian Network separates BELIEF ESTIMATION from ACTION SELECTION (handled by MDP).
- Both prior knowledge (base rates) AND observed evidence correctly update the belief state.
- Naive Bayes assumption (independence of SoundCue and ScreenVisibility) simplifies computation.

2.4. Markov Decision Process — Choose the Optimal Action

Explanation of the Task (Narrative)

After the Bayesian Network estimates threat probabilities, Mario must decide what to do. This is the MDP's role. Mario's situation is described by three factors: danger level (LOW/MEDIUM/HIGH), power-up status (HAS_POWERUP/NO_POWERUP), and location (SAFE_ZONE/NEAR_PIPE/NEAR_LAUNCHER). Five actions are available: ADVANCE, WAIT, JUMP, RETREAT, USE_POWERUP. Each action has uncertain outcomes (transition probabilities) and immediate rewards. Value Iteration computes the optimal value V*(s) for every state and derives the optimal policy pi*(s). When danger is HIGH and Mario has a power-up, USE_POWERUP yields the highest expected return (73.95). When danger is HIGH with no power-up near a launcher, RETREAT is optimal (45.90) — not because retreating is ideal, but because advancing into a Bullet Bill launcher without protection is far worse.

PEAS Description

PEAS Component	Description
Performance	Maximize long-term expected reward: survival + progress + item collection - time cost.
Environment	Partially observable castle section; states characterised by danger level, power-up status, location.
Actuators	ADVANCE, WAIT, JUMP, RETREAT, USE_POWERUP.
Sensors	Current danger level (LOW/MEDIUM/HIGH), power-up availability, location (SAFE/NEAR_PIPE/NEAR_LAUNCHER).

Task Environment Properties

Property	Value	Justification
Observability	Partially Observable	Exact threat positions are not known; inferred from Bayesian Network.
Determinism	Stochastic	Actions have probabilistic outcomes (e.g., ADVANCE succeeds 85% in LOW danger).
Episodicity	Sequential	Each action changes state; future rewards depend on current choices.
Dynamism	Dynamic	Danger level and location change as Mario moves through the level.
Continuity	Discrete	18 finite states, 5 actions, discrete transitions.
Agent Count	Single-Agent	Only Mario is making decisions.

Problem Formulation

State Space S	18 states = 3 danger levels × 2 power-up statuses × 3 locations
Action Space A	ADVANCE, WAIT, JUMP, RETREAT, USE_POWERUP
Transitions T	T(s, a, s') = probability of reaching s' from s via action a
Rewards R	R(s,a,s'): survival=+30, progress=+10, death=-40, time cost=-1 to -5

Discount γ	0.90 — future rewards count but are slightly discounted
Bellman Equation	V(s) = max_a [Σ_{s'} T(s,a,s') × (R(s,a,s') + γ × V(s'))]
Algorithm	Value Iteration — repeat until max |V_new(s) - V(s)| < θ = 1e-6
Convergence	Reached after 116 iterations

Explanation of Algorithm Utilized

A Markov Decision Process (MDP) is a mathematical framework for sequential decision-making under uncertainty. It consists of: a finite set of states S, a finite set of actions A, transition probabilities T(s, a, s'), an immediate reward function R(s, a, s'), and a discount factor $\gamma \in [0,1)$.

The **Markov Property** states that future states depend only on the current state and action, not on history. This makes MDPs computationally tractable for planning under uncertainty.

Value Iteration solves the MDP by repeatedly applying the Bellman Optimality Equation: $V(s) = \max_a \Sigma T(s,a,s')[R(s,a,s') + \gamma V(s')]$. Starting from V(s) = 0 for all states, each iteration improves estimates until convergence (max change < θ). The optimal policy is then $\pi^*(s) = \text{argmax}_a Q(s,a)$, selecting the action with the highest expected cumulative reward.

How the Algorithm is Applied to this Task

The 18-state MDP (3 danger × 2 power-up × 3 location) is solved using Value Iteration with γ=0.9. Transition probabilities encode the physics of each action: ADVANCE in LOW danger succeeds 85% of the time; ADVANCE in HIGH danger with no power-up near a launcher only succeeds 30% and fails catastrophically 30%.

After 116 iterations, Value Iteration converges and produces a complete optimal policy. The key insights: in HIGH danger with HAS_POWERUP, USE_POWERUP always dominates (V*=73.95). In HIGH danger with NO_POWERUP near threats, RETREAT is optimal (V*=45.90). In LOW danger, ADVANCE or JUMP near pipes/launchers.

This MDP framework naturally integrates with the Bayesian Network: the belief state (estimated danger level) from the BN directly maps to an MDP state, creating a complete perception-inference-action pipeline for Mario's AI.

Code

greedy_bfs.py / genetic_algorithm.py / bayesian_network.py / markov_decision_process.py

```
STATES = [(d,p,l) for d in DANGER for p in POWERUP for l in LOCATION]
GAMMA = 0.90; THETA = 1e-6
def get_transitions(state, action): # returns [(prob, next_state, reward)]
danger, powerup, location = state
if action == 'ADVANCE' and danger == 'LOW':
return [(0.85, ('LOW', powerup, 'SAFE_ZONE'), +10),
(0.10, ('MEDIUM', powerup, 'NEAR_PIPE'), -5),
(0.05, ('HIGH', powerup, 'NEAR_LAUNCHER'), -15)]
# ... similar for other actions and danger levels
def value_iteration():
V = {s: 0.0 for s in STATES}
for iteration in range(MAX_ITER):
delta = 0.0
for state in STATES:
action_values = {}
for action in ACTIONS:
```

```
trans = get_transitions(state, action)
q = sum(p*(r + GAMMA*V[ns]) for p,ns,r in trans)
action_values[action] = q
V[state] = max(action_values.values())
policy[state] = max(action_values, key=action_values.get)
delta = max(delta, abs(V[state] - old_V))
if delta < THETA: break # converged
return V, policy
```

Sample Output

Console Output

```
================================================================
MDP Value Iteration — Optimal Action Policy for Mario
================================================================
Iteration 50 | Max delta: 0.03390000
Iteration 100 | Max delta: 0.00020000
Iteration 116 | Max delta: 0.00000000 => Converged!
----------------------------------------------------------------
State Action V*(s)
----------------------------------------------------------------
LOW|HAS_POWERUP|SAFE_ZONE ADVANCE 72.46
LOW|HAS_POWERUP|NEAR_PIPE JUMP 75.14
MEDIUM|HAS_POWERUP|NEAR_PIPE USE_POWERUP 71.68
HIGH|HAS_POWERUP|NEAR_PIPE USE_POWERUP 73.95
HIGH|NO_POWERUP|NEAR_PIPE RETREAT 45.90
HIGH|NO_POWERUP|NEAR_LAUNCHER RETREAT 45.90
Query: Danger=HIGH, PowerUp=HAS_POWERUP, Location=NEAR_PIPE
=> Recommended Action : USE_POWERUP (V*=73.95)
Q-values: USE_POWERUP:73.95, WAIT:62.22, RETREAT:56.70, JUMP:47.57
```

Key Observations

- MDP optimises LONG-TERM expected reward, not just immediate gain — distinguishing it from greedy approaches.
- Value Iteration converged in 116 iterations — confirming finite convergence for bounded reward functions.
- USE_POWERUP dominates in HIGH-danger states when power-up is available — the discount factor captures long-term safety.
- RETREAT is optimal in HIGH-danger + NO_POWERUP near threats (not a failure — it preserves future options).
- The Bellman equation $V(s) = \max_a \Sigma T(s,a,s')[R + \gamma V(s')]$ is the theoretical foundation of optimal control.
- Discount γ=0.9 means future rewards are valued at 90% per step — balancing immediate and future concerns.

2.5. Breadth-First Search — Guarantee the Shortest Route

Explanation of the Task (Narrative)

When Mario has enough time and every move has the same cost, he should not merely choose the cell that looks closest. Breadth-First Search explores the castle level by level. The first time it reaches the Super Star, the route is guaranteed to use the fewest moves.

PEAS Description

PEAS Component	Description
Performance	Reach the Super Star using the minimum number of moves.
Environment	The same 6x7 discrete castle grid with walls and open cells.
Actuators	Move Up, Down, Left, or Right to an adjacent open cell.
Sensors	Current cell, grid layout, visited set, and queue contents.

Task Environment Properties

Property	Value	Justification
Observability	Fully Observable	The complete grid is known before search.
Determinism	Deterministic	Every legal move has a predictable result.
Episodicity	Sequential	Each move changes the available future positions.
Dynamism	Static	Walls and goal remain unchanged during planning.
Continuity	Discrete	The state and action spaces are finite.
Agent Count	Single-Agent	Only Mario is planning a route.

Problem Formulation

State	(row, col) position in the grid
Initial State	(0, 0)
Goal State	(5, 6), the Super Star cell
Frontier	FIFO queue
Guarantee	First goal found is shortest when all moves cost 1
Complexity	O(b^d) time and space

Explanation of Algorithm Utilized

BFS is an uninformed search method. It expands all states at depth 0, then depth 1, then depth 2, and so on. A FIFO queue preserves this order.

Because every Mario move costs one step, the shallowest goal is also the shortest route. BFS is complete and optimal for this uniform-cost grid, but its memory requirement grows quickly on large levels.

How the Algorithm is Applied to this Task

The implementation starts with (0,0) in a deque and records each cell when it is first discovered. Parent paths are stored directly for clarity.

On the supplied level, BFS returns an 11-move route. It is the best baseline for comparing GBFS and A*: GBFS may be faster, while A* aims to reach the same shortest route with fewer unnecessary expansions.

Code

greedy_bfs.py / genetic_algorithm.py / bayesian_network.py / markov_decision_process.py

```
from collections import deque
def breadth_first_search(grid, start, goal):
queue = deque([(start, [start])])
visited = {start}
while queue:
current, path = queue.popleft()
if current == goal:
return path
for neighbor in open_neighbors(current):
if neighbor not in visited:
visited.add(neighbor)
queue.append((neighbor, path + [neighbor]))
return []
```

Sample Output

Console Output

```
Breadth-First Search — Shortest Safe Route
Start: (0, 0) | Goal: (5, 6)
Nodes expanded: 30
Shortest path length: 11 moves
Path reaches the Super Star successfully.
```

Key Observations

- BFS is complete and guarantees the shortest route for equal-cost moves.
- It provides a reliable reference result for the informed search algorithms.
- Its main weakness is memory use because every frontier layer is retained.

2.6. A* Search — Balance Cost and Distance

Explanation of the Task (Narrative)

A* gives Mario a more balanced route planner. Instead of using only the estimated distance to the Super Star, it combines the distance already travelled with the estimated distance remaining. This lets Mario avoid wasted detours while preserving the shortest-path guarantee when the heuristic is admissible.

PEAS Description

PEAS Component	Description
Performance	Reach the Super Star with minimum movement cost and limited exploration.
Environment	The same deterministic castle grid used by BFS and GBFS.
Actuators	Move Up, Down, Left, or Right.
Sensors	Current cell, path cost g(n), Manhattan estimate h(n), and f(n).

Task Environment Properties

Property	Value	Justification
Observability	Fully Observable	The grid and target are available.
Determinism	Deterministic	Each open-cell move costs one.
Episodicity	Sequential	Earlier choices influence path cost.
Dynamism	Static	The search map does not change.
Continuity	Discrete	Cells and actions are discrete.
Agent Count	Single-Agent	Mario is the only planner.

Problem Formulation

State	(row, col) position
Path Cost g(n)	Number of moves taken so far
Heuristic h(n)	Manhattan distance to the goal
Evaluation f(n)	f(n) = g(n) + h(n)
Data Structure	Priority queue ordered by f(n)
Guarantee	Optimal with an admissible, consistent heuristic

Explanation of Algorithm Utilized

A* is an informed best-first search algorithm. It prioritises the state with the smallest f(n)=g(n)+h(n), so it considers both actual cost and estimated remaining cost.

For a four-direction grid with unit movement costs, Manhattan distance never overestimates the remaining cost. Therefore A* is complete and optimal, while usually expanding fewer irrelevant cells than BFS.

How the Algorithm is Applied to this Task

Mario starts with g=0. Each neighbour receives a new cost of g+1 and a Manhattan estimate to (5,6). The priority queue always selects the lowest combined score.

The result is an 11-move shortest path with focused exploration. A* is therefore a strong default when the level is known and speed plus route quality both matter.

Code

greedy_bfs.py / genetic_algorithm.py / bayesian_network.py / markov_decision_process.py

```
import heapq
def a_star_search(grid, start, goal):
open_list = [(heuristic(start, goal), 0, start, [start])]
best_cost = {start: 0}
while open_list:
f, cost, current, path = heapq.heappop(open_list)
if current == goal:
return path
for neighbor in open_neighbors(current):
new_cost = cost + 1
if new_cost < best_cost.get(neighbor, inf):
best_cost[neighbor] = new_cost
heapq.heappush(open_list,
(new_cost + heuristic(neighbor, goal),
new_cost, neighbor, path + [neighbor]))
```

Sample Output

Console Output

```
A* Search — Efficient Shortest Route
Start: (0, 0) | Goal: (5, 6)
Nodes expanded: 21
Optimal path length: 11 moves
f(n) = g(n) + h(n) guided the search.
```

Key Observations

- A* combines the reliability of BFS with the direction of a heuristic.
- Manhattan distance is admissible here, so the 11-move route is optimal.
- A* is usually preferable to GBFS when route quality cannot be sacrificed.

2.7. Particle Swarm Optimization — Tune Mario's Policy

Explanation of the Task (Narrative)

Instead of evolving Mario's behaviour through genetic operators, I also modelled Mario as a swarm-optimisation problem. Each particle represents a possible set of behavioural weights. Particles remember their own best policy and are attracted toward the best policy discovered by the swarm.

PEAS Description

PEAS Component	Description
Performance	Maximise completion, survival, coins, power-ups, and time efficiency.
Environment	A simulated Mario level with enemies, rewards, and risky jumps.
Actuators	Continuous changes to seven behavioural policy weights.
Sensors	Current particle position, velocity, personal best, and global best.

Task Environment Properties

Property	Value	Justification
Observability	Partially Observable	Fitness summarises outcomes rather than every hidden event.
Determinism	Stochastic	Random initial particles and velocity updates are used.
Episodicity	Sequential	Iterations improve the best policy over time.
Dynamism	Dynamic	The swarm changes its candidate policies each iteration.
Continuity	Continuous	Each policy dimension is a real value in [0,1].
Agent Count	Multi-Agent	Particles cooperate indirectly as a swarm.

Problem Formulation

Particle	A seven-dimensional candidate Mario policy
Position	[coin, power-up, avoidance, jump, goal, retreat, space]
Velocity	Direction and magnitude of each policy update
Fitness	Composite score for completion, rewards, and survival
Update Rule	Inertia + personal attraction + social attraction
Bounds	Each policy weight is clipped to [0,1]

Explanation of Algorithm Utilized

PSO is a population-based continuous optimisation technique inspired by the movement of birds or fish. Every particle moves through the search space using its velocity and learns from two sources: its own best position and the swarm's global best position.

Unlike GA, PSO does not require crossover or mutation. Its simple social update can converge quickly, but it can also converge prematurely if the swarm loses diversity or the fitness function is poorly designed.

How the Algorithm is Applied to this Task

Each of the 20 particles starts with seven random weights. The fitness function is kept comparable with the GA experiment, rewarding progress, coins, power-ups, survival, and safe movement.

After 30 iterations, the swarm reports its best policy. This makes PSO a useful alternative when Mario's control parameters are continuous and a good policy can be evaluated without differentiable mathematics.

Code

greedy_bfs.py / genetic_algorithm.py / bayesian_network.py / markov_decision_process.py

```
for particle in swarm:
for d in range(DIMENSIONS):
velocity[d] = (INERTIA * velocity[d]
+ COGNITIVE * r1 * (personal_best[d] - position[d])
+ SOCIAL * r2 * (global_best[d] - position[d]))
position[d] = clip(position[d] + velocity[d], 0.0, 1.0)
score = fitness(position)
update_personal_and_global_bests(particle, score)
```

Sample Output

Console Output

```
Particle Swarm Optimization — Tuning Mario's Policy
Particles: 20 | Iterations: 30
Best fitness: 680.00
GoalUrgency: 1.000 | EnemyAvoidance: 1.000
JumpAggressiveness: 1.000 | OpenSpacePreference: 1.000
```

Key Observations

- PSO found a high-scoring policy by sharing information across particles.
- The result is sensitive to inertia, learning coefficients, and fitness design.
- PSO complements GA: both optimise policies, but they explore the search space differently.

3. Comparative Analysis & Discussion

Algorithm Comparison Table

Aspect	Greedy BFS	Genetic Algo	Bayesian Net	MDP
Problem Type	Path search	Optimisation	Probabilistic inference	Sequential decision
Optimal?	No	Approximate	Yes (given model)	Yes (exact)
Handles Uncertainty?	No	Partially (stochastic fitness)	Yes (core purpose)	Yes (transition probs)
Mario Task	Chase Super Star	Evolve playing policy	Infer hidden threats	Choose best action
Key Strength	Fast, goal-directed	Finds non-obvious solutions	Reasons under partial observability	Long-term optimal planning
Key Weakness	Not optimal, can fail with bad heuristic	Needs fitness function design, slow convergence	Requires accurate prior probabilities	State space explosion with large problems
Complexity	O(b^m) time	O(G×P×F)	O(2^n) exact; linear with independence assumptions	O(iter × |S| × |A| × |S|)

Discussion: Why These Seven Algorithms Complement Each Other

These seven algorithms address different phases of Mario's intelligence. **Greedy BFS** solves immediate navigation when speed matters and a heuristic is available. It is cheap to implement and works well for time-sensitive objectives like chasing a disappearing star. However, it cannot handle uncertainty or optimise long-term behaviour.

The **Genetic Algorithm** operates at a higher level — it does not plan a specific path but evolves a general playing style. This is valuable when the solution space is complex, non-linear, and the objective involves multiple competing criteria (coins vs. survival vs. level completion). The GA's population-based search avoids local optima that single-trajectory methods like hill-climbing would get trapped in.

BFS provides a dependable shortest-path baseline, while **A*** uses the same guarantee with a heuristic that focuses exploration. The **Particle Swarm Optimization** experiment offers another way to tune continuous behaviour weights, complementing the Genetic Algorithm's selection, crossover, and mutation approach.

The **Bayesian Network** addresses partial observability — the most realistic feature of the game. Mario cannot see everything; he must reason about hidden states from indirect evidence. The BN separates belief estimation from action selection, creating a clean and interpretable probabilistic model that updates correctly as new evidence arrives.

The **MDP** completes the intelligence pipeline by converting beliefs into optimal actions. Where GBFS is myopic (next-step only) and the GA is offline (policy computed before playing), the MDP reasons explicitly about future consequences and uncertainty. The combination of BN + MDP creates a full perception-inference-action loop for playing under uncertainty.

Reflective Questions

Q1: Greedy BFS found the Super Star in 13 expansions. BFS would guarantee the shortest path — but would take more expansions. When is Greedy BFS preferred despite being suboptimal?

Answer: When time is limited (star about to vanish), a fast good-enough solution beats an optimal slow solution. Greedy BFS is preferred when the heuristic is reliable and the cost of delay outweighs the cost of a longer path.

Q2: The Genetic Algorithm kept JumpAggressiveness low (0.169) even though other traits were maximised. What does this tell us about the fitness landscape?

Answer: High JumpAggressiveness without corresponding OpenSpacePreference causes falls. The GA discovered this non-obvious interaction through evolution — without any programmer explicitly encoding this relationship in the code.

Q3: If P(PipeThreat) = 20%, should Mario always advance?

Answer: Not necessarily. The consequence of a Piranha Plant hit (death, losing a life) is severe. Even a 20% chance of a catastrophic outcome may justify waiting or retreating — especially if the alternative (waiting) has low cost. The MDP captures this by weighting consequences, not just probabilities.

Q4: The MDP recommended RETREAT when danger=HIGH and no power-up near launcher. Does this mean retreating is ideal?

Answer: No — it means retreating has the best EXPECTED LONG-TERM VALUE given the current constraints. It preserves Mario's life for a future attempt with better resources. In MDP, optimal ≠ locally best; it means globally best over time.

4. Conclusion

Result Analysis

This case study demonstrated how seven fundamentally different AI algorithms can each contribute to making Mario an intelligent agent. The results were both technically sound and practically insightful:

- **Greedy Best-First Search** successfully guided Mario to the Super Star in just 13 node expansions, finding an 11-move path. The algorithm's speed makes it ideal for time-sensitive objectives, though it is not guaranteed to find the shortest path and can fail if the heuristic is misleading.
- **Genetic Algorithm** evolved a high-fitness Mario policy (fitness: 680.00) over 30 generations — a 47% improvement from the initial random population (462.03). The evolved policy correctly balanced GoalUrgency and EnemyAvoidance while learning to keep JumpAggressiveness low — a non-obvious interaction that no programmer manually encoded.
- **Bayesian Network** demonstrated that observable clues significantly update belief: P(PipeThreat) rose from a prior of 0.35 to a posterior of 0.72 when both sound and partial visibility were detected. This correctly shifted the recommendation from ADVANCE to WAIT — showing rational reasoning under partial observability.
- **Markov Decision Process** produced a complete optimal policy across all 18 states after 116 Value Iteration steps. The policy correctly identified USE_POWERUP (V*=73.95) as optimal in high-danger + power-up states, and RETREAT (V*=45.90) as optimal in high-danger + no power-up near launchers — prioritising long-term survival over immediate progress.

Broader Reflections

Working on this case study helped me understand something important about AI — no single algorithm is the 'best'. Each method I implemented has a specific strength that becomes valuable in the right context. BFS is reliable, A* balances route quality with speed, and Greedy BFS is intuitive and fast but sacrifices correctness. Genetic Algorithms and PSO are powerful optimisers but require careful fitness function design to avoid rewarding wrong behaviour. Bayesian Networks make reasoning under uncertainty mathematically principled, while MDPs complete the decision cycle by translating beliefs into actions with long-term consequences.

The Super Mario domain was particularly effective as a teaching tool because every algorithm connected to a concrete, relatable scenario. This made it easy to intuitively verify whether a result made sense — a policy that chases coins into dangerous areas clearly has a poorly designed fitness function, and the MDP clearly should not recommend advancing into a Bullet Bill launcher without protection. The game environment naturally enforced sanity-checking of every algorithmic result.

Looking forward, the combination of Bayesian Networks + MDPs (a Partially Observable MDP or POMDP) represents one of the most powerful frameworks in modern AI for real-world decision-making — used in robotics, medical diagnosis, autonomous driving, and game-playing agents. Starting with these building blocks in the Mario domain provides an excellent foundation for understanding more advanced AI systems.

5. References

[1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. — Primary reference for all search algorithms, Bayesian Networks, and MDPs covered in this case study.

[2] Mitchell, M. (1998). *An Introduction to Genetic Algorithms*. MIT Press. — Theoretical foundation for chromosome representation, fitness functions, crossover and mutation operators.

[3] Koller, D., & Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press. — In-depth treatment of Bayesian Networks, conditional independence, and inference algorithms.

[4] Puterman, M. L. (2014). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley-Interscience. — Formal MDP theory, Bellman equations, and Value Iteration convergence proofs.

[5] Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann. — Original formulation of belief networks and Bayesian inference.

[6] Togelius, J., Yannakakis, G. N., Stanley, K. O., & Browne, C. (2011). Search-based Procedural Content Generation: A Taxonomy and Survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3), 172-186. — Application of evolutionary algorithms and search in game environments.

[7] Kaelbling, L. P., Littman, M. L., & Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2), 99-134. — Foundation for POMDP framework combining Bayesian Networks and MDPs.

Siddhant Kumar Singh | Roll No: 2024109296

Case Study in Artificial Intelligence — Super Mario AI

Algorithms: BFS | A* | Greedy BFS | GA | PSO | Bayesian Network | MDP